



PayNexus

Uma carteira digital distribuída em microserviços

Fintech

Blockchain

Microservices



O que é & que problema resolve

Os utilizadores autenticam-se, carregam a carteira com um pagamento real por cartão, movimentam fundos como transações *on-chain* e são notificados em tempo real — tudo entregue como serviços independentes.



O problema

Um monólito que mistura autenticação, pagamentos, blockchain e notificações é difícil de construir, escalar e operar — cada parte tem linguagens, escalas e falhas diferentes.

A solução

Dividir em serviços com fronteiras bem definidas (bounded contexts), cada um dono dos seus dados e do seu ciclo de vida. A arquitetura é, ela própria, o deliverable do projeto.

O projeto num relance



3
Microserviços
IAM · Payments · Notifications



4
Linguagens
Python · Java · Go · TS



~10
Hosts
Kubernetes



1
API Gateway
Traefik · só porto :80

Domínio	Carteira digital · pagamentos · transações blockchain	Arquitetura	Microserviços atrás de um único API gateway
Autenticação	Keycloak (OAuth2 / OIDC) · 2 realms	Pagamentos	Stripe (PaymentIntent + webhooks) + Twilio OTP
Blockchain	Ethereum Sepolia testnet via Web3j	Tempo real	Web Push notifications (multi-tenant)
Dados	PostgreSQL por serviço + Redis cache	Secrets	HashiCorp Vault (AppRole + agent sidecars)

Porquê microserviços



Bounded contexts

Cada serviço é dono de um domínio: identidade, pagamentos, transações, notificações.



Deploy independente

Cada serviço com o seu Dockerfile, imagem e ciclo de vida próprios.



Poliglota

A melhor ferramenta para cada caso — Python, Java/Spring, Go.



DB por serviço

Sem schema partilhado; uma instância PostgreSQL por serviço, em redes isoladas.



Isolamento de falhas

Uma falha nas notificações não derruba os pagamentos.



Ponto de entrada único

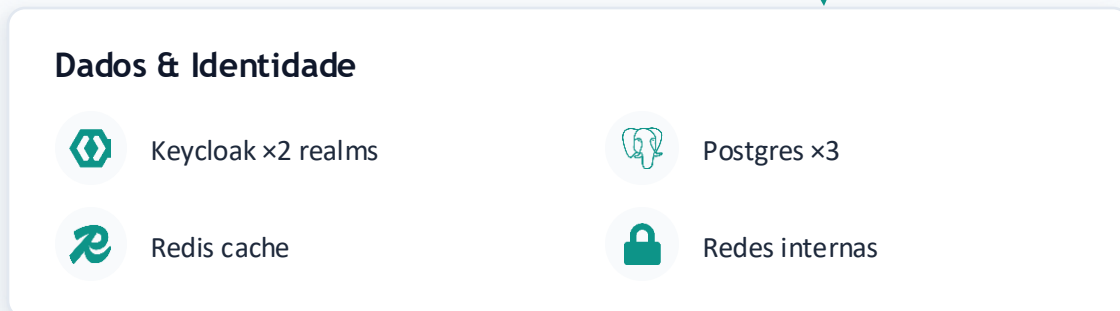
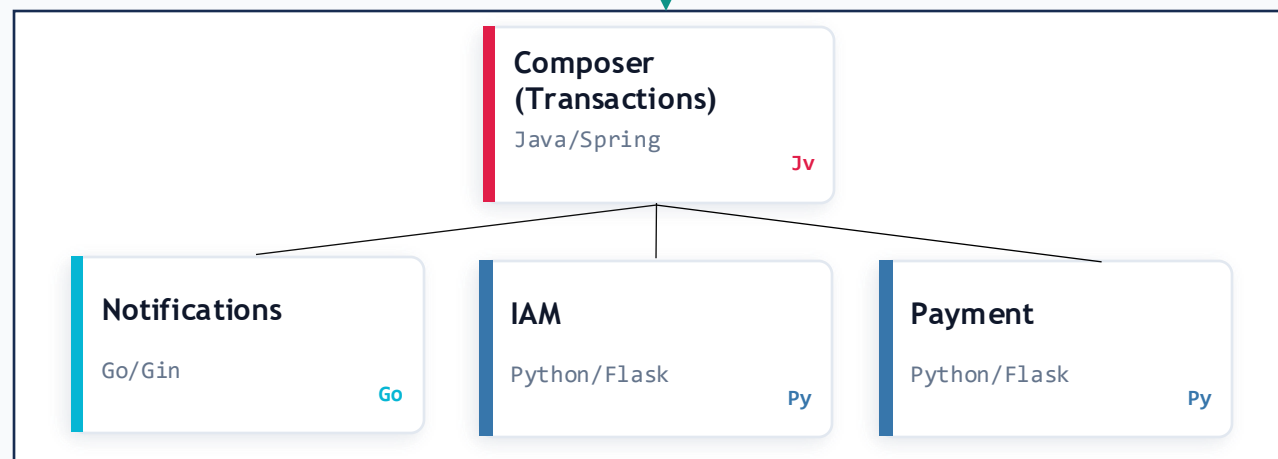
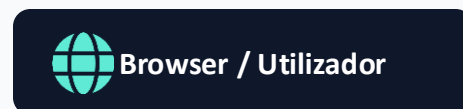
O gateway Traefik esconde a topologia interna atrás de um só porto.



Trade-offs aceites: mais complexidade operacional (muitos containers e secrets), consistência eventual entre serviços e auth cross-service que um monólito não precisaria.

Arquitetura do sistema

Fluxo vertical: cliente → gateway → serviços → dados



Catálogo de microsserviços

Serviço	Responsabilidade	Linguagem	Datastore
iam_service	Sign-up, login, validação de tokens via Keycloak	Python / Flask	Keycloak
payment_service	Pagamentos por cartão, OTP, recibos, cartões guardados	Python / Flask	PostgreSQL
payment frontend	UI de pagamento dedicada	React + TS + MUI	—
notifications_service	Clientes, subscrições, WebPush em tempo real, envio de emails	Go / Gin	Postgres + Redis
frontend	UI principal de carteira & transações	React + TS + Vite	—
composer	Carteiras, transações, composição (BFF)	Java / Spring	PostgreSQL

BFF = Backend-for-Frontend: Transactions agrega trabalho e dispara as notificações.

Stack tecnológica

Frontend

-  React 18 + Vite
-  TypeScript
-  MUI · Axios · React Query

Backend

-  Python + Flask
-  Java 21 + Spring Boot
-  Go + Gin

Dados

-  PostgreSQL 15
-  Redis 7
-  SQLAlchemy · JPA · GORM

Auth & Gateway

-  Keycloak 26 · OIDC
-  Traefik v3.1
-  JWT · 2 realms

Plataforma

-  Docker + Compose
-  Kubernetes (DETI)
-  HashiCorp Vault 1.17

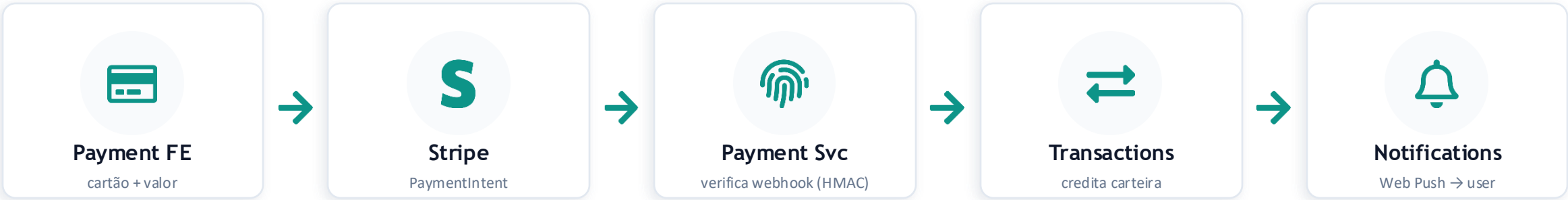
Integrações & Obs.

-  Stripe
-  Web3j · Sepolia
-  Grafana 10

Comunicação entre serviços

REST/HTTP na rede interna · dois estilos de auth: **JWT Bearer** (utilizador) e **X-Internal-Key** (serviço-a-serviço).

Fluxo end-to-end — “carregar & notificar”



Chamada	De → Para	Endpoint	Auth
Concluir pagamento	Payment → Transactions	PATCH /v1/payments/{id}	X-Internal-Key
Emitir evento	Transactions → Notifications	POST /v1/events	Bearer <api-key>
Login / token	Frontend → IAM → Keycloak	OIDC auth-code flow	OAuth2
Entrega push	Notifications → Browser	Web Push (encrypted)	VAPID keys

Autenticação & segurança

Keycloak · 2 realms

Isolamento entre utilizadores da carteira e dos pagamentos.



microservices realm

utilizadores da carteira



payments realm

utilizadores de pagamentos

OIDC auth-code → JWT em cada chamada protegida

Login

Keycloak

Tokens (JWT)

Bearer →



JWT Bearer

authn em endpoints protegidos



RBAC (operator)

endpoints só de admin



Header X-Internal-Key

confiança serviço-a-serviço



Stripe HMAC

verifica webhooks da Stripe



HashiCorp Vault

sem secrets no código/imagens



Carteiras AES

chaves privadas encriptadas

Pagamentos com Stripe



Destaques

- ✓ **Confirmação** estado concluded só via webhook verificado da Stripe
- ✓ **Forte autent.** OTP por WhatsApp (Twilio) antes de concluir
- ✓ **Recibos** PDF gerados com ReportLab
- ✓ **Segurança** verificação HMAC no header Stripe-Signature
- ✓ **Cartões guardados** associados a um Stripe Customer para reutilização

Blockchain & notificações



Blockchain & Transações

O serviço Transactions (Java/Spring) é onde o dinheiro se torna valor on-chain.

- ✓ **Web3j 4.10** biblioteca de acesso à chain
- ✓ **Sepolia testnet** Ethereum · chain id 11155111
- ✓ **Carteiras por user** chaves privadas encriptadas com AES
- ✓ **Operações** saldo · estimativa de gas · enviar transação
- ✓ **Providers** mock (dev) e real (produção)



Notificações (tempo real)

Motor de Web Push white-label e multi-tenant: qualquer cliente é onboarded em runtime.

- ✓ **Onboarding** cliente em runtime → API key + VAPID keys
- ✓ **Entrega** Web Push no browser via Service Worker
- ✓ **Multi-tenant** cada cliente com o seu par VAPID (branding)
- ✓ **Performance** Redis em cache de clientes/subscrições
- ✓ **API** POST /v1/events/subscribe e POST /v1/events

Docker & Kubernetes



Containerização

Cada serviço como imagem própria, com Dockerfiles multi-stage (build + runtime slim).

~17 containers via Docker Compose em dev/demo.



Isolamento de rede

egs-network serviços públicos + Traefik

transactions-db-net transactions + Postgres

payment-db-net payment + Postgres

notifications-db-net notifications + Postgres + Redis

	Docker Compose	Kubernetes
Uso	Dev local / demo	Produção (cluster)
Escala	Manual	Réplicas, declarativo
Routing	Container Traefik	Traefik Ingress
Secrets	Vault agents → .env	K8s Secrets
Storage	Volumes nomeados	PersistentVolumeClaims
Âmbito	~17 containers, 1 host	Namespaced, multi-nó

Produção: cluster DETI • namespace tenant-grupo3-egs-deti-ua-pt

Observabilidade & execução



Observabilidade · Grafana 10

Datasources: as 3 bases PostgreSQL (transactions, payments, notifications).

main_dashboard

Visão de sistema: transações feitas, notificações enviadas, pagamentos concluídos.

payments_dashboard, notifications_dashboard, transactions_dashboard

Dashboards com dados detalhados para um melhor controlo e gestão da plataforma



URLs

<https://transactions-frontend.grupo3-egs-deti-ua.pt>

<https://payment.grupo3-egs-deti-ua.pt>

<https://keycloak.grupo3-egs-deti-ua.pt>

<http://grafana.grupo3-egs-deti-ua.pt>

Desafios & lições aprendidas



Ops poliglota

4 linguagens = 4 toolchains de build/runtime para containerizar e manter consistentes.



Secrets à escala

Vault + sidecars tiraram secrets do código, mas adicionaram peças móveis.



Dois realms Keycloak

Forte isolamento entre users de carteira e pagamentos, ao custo de duplicação de config.



Pagamentos por webhook

Confiar no webhook da Stripe (não no cliente) implica abraçar consistência eventual.



Testnet blockchain

Transações reais na Sepolia exigem gas, encriptação de chaves e provider mock para dev.



Gateway único

Traefik simplificou a superfície — um porto exposto, tudo o resto interno.

Obrigado.

Perguntas?

3

microserviços

4

linguagens

1

gateway

10

Hosts

Uma carteira digital distribuída — pagamentos reais, valor on-chain e notificações em tempo real, entregues como serviços independentes.